

Effective prompt engineering is crucial to getting the desired performance from large language models (LLMs) like Command R/R+. This process can be time-consuming, especially for complex tasks or when comparing models. To ensure fair comparisons and optimize performance, it's essential to use the correct special tokens, which may vary between models and significantly impact outcomes.

Each task requires its own prompt template. This document outlines the structure and best practices for the following use cases:

- Retrieval-Augmented Generation (RAG) with Command R/R+
- Summarization with Command R/R+
- Single-Step Tool Use with Command R/R+ (Function Calling)
- Multi-Step Tool Use with Command R/R+ (Agents)

The easiest way to make sure your prompts will work well with Command R/R+ is to use our [tokenizer on Hugging Face](#) . Today, HuggingFace has prompt templates for Retrieval-Augmented Generation (RAG) and Single-Step Tool Use with Command R/R+ (Function Calling). We are working on adding prompt templates in HuggingFace for Multi-Step Tool Use with Command R/R+ (Agents).

High-Level Overview of Prompt Templates

The prompt for Command R/R+ is composed of structured sections, each serving a specific purpose. Below is an overview of the main components. We've color coded the different sections of the prompt to make them easy to pick out and we will go over them in more detail later.

Augmented Generation Prompt Template (RAG and Summarization)

In RAG, the workflow involves two steps:

1. **Retrieval:** Retrieving the relevant snippets.
2. **Augmented Generation:** Generating a response based on these snippets.

Summarization is very similar to augmented generation: the model takes in some documents and its response (the summary) needs to be conditioned on those documents.

This way, RAG and Summarization follow a similar prompt template. It is the Augmented Generation prompt template and here's what it looks like at a high level:

```
augmented_gen_prompt_template =
```

```
"""<BOS_TOKEN></START_OF_TURN_TOKEN></SYSTEM_TOKEN># Safety
```

```
Preamble
```

```
{SAFETY_PREAMBLE}
```

```
# System Preamble
```

```
## Basic Rules
```

```
{BASIC_RULES}
```

```
# User Preamble
```

```
## Task and Context
```

```
{TASK_CONTEXT}
```

```
## Style Guide
```

```
{STYLE_GUIDE}</END_OF_TURN_TOKEN>{CHAT_HISTORY}
```

```
</START_OF_TURN_TOKEN></SYSTEM_TOKEN>{RETRIEVED_SNIPPETS_FOR_RAG  
or TEXT_TO_SUMMARIZE}</END_OF_TURN_TOKEN></START_OF_TURN_TOKEN>
```

```
</SYSTEM_TOKEN>{INSTRUCTIONS}</END_OF_TURN_TOKEN>
```

```
</START_OF_TURN_TOKEN></CHATBOT_TOKEN>"""
```

We can see that the prompt is set up in a structured way where we have sections for things like the **basic rules** we want the model to follow, the **task** we want it to solve, and the **style** in which it should write its output in.

Single step Tool Use with Command R/R+ (Function Calling)

Single-step tool use (or “Function Calling”) allows Command R/R+ to interact with external tools like APIs, databases, or search engines. Single-step tool use is made of two model inferences:

1. **Tool Selection:** The model decides which tools to call and with what parameters. It’s then up to the developer to execute these tool calls and obtain tool results.
2. **Response Generation:** The model generates the final response given the tool results.

You can learn more about single step tool use [in our documentation](#). Let’s go over the prompt template for Tool Section, and for Response Generation.

A) Tool Selection Prompt Template

```
singlestep_tool_selection_prompt_template =
```

```
"""<BOS_TOKEN></START_OF_TURN_TOKEN></SYSTEM_TOKEN># Safety
```

```
Preamble
```

```
{SAFETY_PREAMBLE}
```

System Preamble

Basic Rules

{BASIC_RULES}

User Preamble

Task and Context

{TASK_CONTEXT}

Style Guide

{STYLE_GUIDE}

Available Tools

**{TOOLS}</END_OF_TURN_TOKEN/>{CHAT_HISTORY}</START_OF_TURN_TOKEN/>
</SYSTEM_TOKEN/>{INSTRUCTIONS_FOR_SINGLE_STEP_TOOL_USE}
</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/></CHATBOT_TOKEN/>""**

The prompt template for Tool Selection is similar to the Augmented Generation prompt template. There are, however, two spots that differ which are highlighted. The first is that we have added the tool definitions which come after the style guide (you can see that there's now an **## Available Tools** section), and the second is that we've removed the section with the retrieved snippets or text to summarize.

B) Response Generation Template

At this point, Command R/R+ has decided which tools to call and with what parameters (see previous section). Developers are expected to execute these tool calls, and to receive tool results in return.

In this Response Generation step, the goal is to generate the final model response, given the tool results. This is another case of... Augmented Generation!

Therefore, the prompt template is very similar to the augmented generation prompt used for RAG and Summarization. The only difference is that we replace the RAG snippets and/or text to summarize with tool outputs (TOOL_OUTPUTS).

singlestep_augmented_generation_prompt_template =

**""<BOS_TOKEN></START_OF_TURN_TOKEN/></SYSTEM_TOKEN/># Safety
Preamble
{SAFETY_PREAMBLE}**

System Preamble

Basic Rules

```
{BASIC_RULES}
```

```
# User Preamble
```

```
## Task and Context
```

```
{TASK_CONTEXT}
```

```
## Style Guide
```

```
{STYLE_GUIDE}<|END_OF_TURN_TOKEN|>{CHAT_HISTORY}
```

```
<|START_OF_TURN_TOKEN|><|SYSTEM_TOKEN|>{TOOL_OUTPUTS}
```

```
<|END_OF_TURN_TOKEN|><|START_OF_TURN_TOKEN|><|SYSTEM_TOKEN|>
```

```
{INSTRUCTIONS_FOR_SINGLE_STEP_TOOL_USE}<|END_OF_TURN_TOKEN|>
```

```
<|START_OF_TURN_TOKEN|><|CHATBOT_TOKEN|>""
```

Multi step Tool Use with Command R/R+ (Agents)

Multi-step tool use is suited for building agents that can plan and execute a sequence of actions using multiple tools. Unlike single-step tool use, the model can perform several inference cycles, iterating through Action → Observation → Reflection until it decides on a final response. For more details, refer to our [documentation on multi-step tool use](#).

To understand the multistep tool use prompt, let's look at the following prompts

- The prompt template for step 1 of the agent
- The prompt template for step 2 of the agent
- The prompt template at step i of the agent

A) Prompt template for Step 1 of the agent

```
multistep_tooluse_step_1_prompt_template =
```

```
""""<BOS_TOKEN><|START_OF_TURN_TOKEN|><|SYSTEM_TOKEN|># Safety
```

```
Preamble
```

```
{SAFETY_PREAMBLE}
```

```
# System Preamble
```

```
## Basic Rules
```

```
{BASIC_RULES}
```

```
# User Preamble
```

```
## Task and Context
```

```
{TASK_CONTEXT}
```

```
## Style Guide
```

```
{STYLE_GUIDE}
```

```
## Available Tools
```

```
{TOOLS}</END_OF_TURN_TOKEN>{CHAT_HISTORY}</START_OF_TURN_TOKEN>  
</SYSTEM_TOKEN>{INSTRUCTIONS_FOR_MULTI_STEP_TOOL_USE}  
</END_OF_TURN_TOKEN></START_OF_TURN_TOKEN></CHATBOT_TOKEN>"""
```

In this first step of the agent, the model generates an initial plan and suggests tool calls. Developers are expected to execute these tool calls, and to receive tool results in return.

B) Prompt template for subsequent steps of the agent

As the process continues to step 2 (or any subsequent step), the model evaluates the tool results from the previous step, self-reflects and updates its plan. It may choose to call additional tools or decide that it has gathered enough information to provide a final response.

This iterative process continues for as many steps as the model deems necessary.

Here is the template for Step 2 of the agent:

```
multistep_tooluse_step_2_prompt_template =  
""""<BOS_TOKEN></START_OF_TURN_TOKEN></SYSTEM_TOKEN># Safety  
Preamble  
{SAFETY_PREAMBLE}  
  
# System Preamble  
## Basic Rules  
{BASIC_RULES}  
  
# User Preamble  
## Task and Context  
{TASK_CONTEXT}  
  
## Style Guide  
{STYLE_GUIDE}  
  
## Available Tools  
{TOOLS}</END_OF_TURN_TOKEN>{CHAT_HISTORY}</START_OF_TURN_TOKEN>  
</SYSTEM_TOKEN>{INSTRUCTIONS_FOR_MULTI_STEP_TOOL_USE}  
</END_OF_TURN_TOKEN></START_OF_TURN_TOKEN></CHATBOT_TOKEN>  
{PLAN_AND_SUGGESTED_TOOL_CALLS_FOR_STEP_1}</END_OF_TURN_TOKEN>
```

```
</START_OF_TURN_TOKEN/></SYSTEM_TOKEN/>{TOOL_RESULTS_FROM_STEP_1}  
</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/></CHATBOT_TOKEN/>""
```

Here is the template for Step i of the agent:

```
multistep_tooluse_step_i_prompt_template =  
""""<BOS_TOKEN></START_OF_TURN_TOKEN/></SYSTEM_TOKEN/># Safety  
Preamble  
{SAFETY_PREAMBLE}  
  
# System Preamble  
## Basic Rules  
{BASIC_RULES}  
  
# User Preamble  
## Task and Context  
{TASK_CONTEXT}  
  
## Style Guide  
{STYLE_GUIDE}  
  
## Available Tools  
{TOOLS}</END_OF_TURN_TOKEN/>{CHAT_HISTORY}</START_OF_TURN_TOKEN/>  
</SYSTEM_TOKEN/>{INSTRUCTIONS_FOR_MULTI_STEP_TOOL_USE}  
</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/></CHATBOT_TOKEN/>  
{PLAN_AND_SUGGESTED_TOOL_CALLS_FOR_STEP_1}</END_OF_TURN_TOKEN/>  
</START_OF_TURN_TOKEN/></SYSTEM_TOKEN/>{TOOL_RESULTS_FROM_STEP_1}  
</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/></CHATBOT_TOKEN/>  
{PLAN_AND_SUGGESTED_TOOL_CALLS_FOR_STEP_2}</END_OF_TURN_TOKEN/>  
</START_OF_TURN_TOKEN/></SYSTEM_TOKEN/>{TOOL_RESULTS_FROM_STEP_2}  
</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/></CHATBOT_TOKEN/>  
{PLAN_AND_SUGGESTED_TOOL_CALLS_FOR_STEP_3}</END_OF_TURN_TOKEN/>  
</START_OF_TURN_TOKEN/></SYSTEM_TOKEN/>{TOOL_RESULTS_FROM_STEP_3}  
</END_OF_TURN_TOKEN/>...etc...</START_OF_TURN_TOKEN/>  
</CHATBOT_TOKEN/>{PLAN_AND_SUGGESTED_TOOL_CALLS_FOR_STEP_i-1}  
</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/></SYSTEM_TOKEN/>  
{TOOL_RESULTS_FROM_STEP_i-1}</END_OF_TURN_TOKEN/>  
</START_OF_TURN_TOKEN/></CHATBOT_TOKEN/>""
```

Detailed Prompt Templates

Now that we have a high-level understanding of prompt templates, let's dive into the detailed prompts for each task.

Augmented Generation: RAG with Command R/R+

Retrieval Augmented Generation (RAG) involves two main steps:

- Retrieval: retrieve the relevant snippets
- Augmented Generation: generate a response based on these snippets.

Below is a detailed look at the fully rendered prompt for Augmented Generation. You can achieve the same result using the Hugging Face Tokenizer's `apply_grounded_generation_template()` function.

The chat history in this example, is the simplest it can be: the user question only.

```
CHAT_HISTORY = "<|START_OF_TURN_TOKEN|><|USER_TOKEN|>Where do the  
tallest penguins live?<|END_OF_TURN_TOKEN|>"
```

The retrieved snippets for RAG should be wrapped in `<|START_OF_TURN_TOKEN|><|SYSTEM_TOKEN|>{RETRIEVED_SNIPPETS_FOR_RAG}<|END_OF_TURN_TOKEN|>` and look something like this:

```
RETRIEVED_SNIPPETS_FOR_RAG =  
"""<results>  
Document: 0  
title: Tall penguins  
snippet: Emperor penguins are the tallest growing up to 122 cm in height.  
  
Document: 1  
title: Penguin habitats  
snippet: Emperor penguins only live in Antarctica.  
</results>"""
```

Each chunk should start with **Document: {n}** and should be an ascending list of integers starting at 0.

Below is a detailed look at the fully rendered prompt for Augmented Generation.

```
RAG_augmented_generation_prompt_template =  
"""<BOS_TOKEN><|START_OF_TURN_TOKEN|><|SYSTEM_TOKEN|># Safety  
Preamble  
The instructions in this section override those in the task description and style guide
```

sections. Don't answer questions that are harmful or immoral

System Preamble

Basic Rules

You are a powerful conversational AI trained by Cohere to help people. You are augmented by a number of tools, and your job is to use and consume the output of these tools to best help the user. You will see a conversation history between yourself and a user, ending with an utterance from the user. You will then see a specific instruction instructing you what kind of response to generate. When you answer the user's requests, you cite your sources in your answers, according to those instructions.

User Preamble

Task and Context

You help people answer their questions and other requests interactively. You will be asked a very wide array of requests on all kinds of topics. You will be equipped with a wide range of search engines or similar tools to help you, which you use to research your answer. You should focus on serving the user's needs as best you can, which will be wide-ranging.

Style Guide

Unless the user asks for a different style of answer, you should answer in full sentences, using proper grammar and spelling.</END_OF_TURN_TOKEN/>

</START_OF_TURN_TOKEN/></USER_TOKEN/>Where do the tallest penguins live?

</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/></SYSTEM_TOKEN/>

<results>

Document: 0

title: Tall penguins

snippet: Emperor penguins are the tallest growing up to 122 cm in height.

Document: 1

title: Penguin habitats

snippet: Emperor penguins only live in Antarctica.

</results></END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/>

</SYSTEM_TOKEN/>Carefully perform the following instructions, in order, starting each with a new line.

Write 'Grounded answer:' followed by a response to the user's last input in high quality natural english. Use square brackets to indicate a citation from the search results, e.g. "my fact [0]" for a fact from document 0.</END_OF_TURN_TOKEN/>

</START_OF_TURN_TOKEN/></CHATBOT_TOKEN/>""""

And this results in the model output:

Grounded answer: The tallest penguins are Emperor penguins [0], which grow up to 122 cm in height. [0] They live only in Antarctica. [1]

Augmented Generation: Summarization with Command R/R+

Summarization is very similar to RAG. The key differences are:

- We want to create a summary of the provided documents
- Unlike the chunks retrieved in RAG, the order of the chunks the model receives actually matters for summarization.

Starting from our augmented generation prompt, we can adapt it a bit by changing the **{TASK_CONTEXT}** to better fit the summarization task.

TASK_CONTEXT = You will receive a series of text fragments from an article that are presented in chronological order. As the assistant, you must generate responses to user's requests based on the information given in the fragments. Ensure that your responses are accurate and truthful, and that you reference your sources where appropriate to answer the queries, regardless of their complexity.

Similar to the previous section, we will use the simplest chat history: just one message from the user.

CHAT_HISTORY = "<|START_OF_TURN_TOKEN|><|USER_TOKEN|>Summarize the documents in 20 words or less<|END_OF_TURN_TOKEN|>"

The text to summarize should be wrapped in **<|START_OF_TURN_TOKEN|><|SYSTEM_TOKEN|>{TEXT_TO_SUMMARIZE}<|END_OF_TURN_TOKEN|>** and look something like this:

TEXT_TO_SUMMARIZE =

""""<results>

Document: 0

title: Tall penguins

snippet: Emperor penguins are the tallest growing up to 122 cm in height.

Document: 1

title: Penguin habitats

snippet: Emperor penguins only live in Antarctica.

</results>""""

We recommend splitting the text to summarize into chunks of 100-250 words. Each chunk should start with **Document: {n}** and should be an ascending list of integers starting at 0.

Below is a detailed look at what the fully rendered prompt looks like for summarization.

```
summarization_augmented_generation_prompt_template =
```

```
"""<BOS_TOKEN></START_OF_TURN_TOKEN></SYSTEM_TOKEN># Safety
```

```
Preamble
```

```
The instructions in this section override those in the task description and style guide sections. Don't answer questions that are harmful or immoral
```

```
# System Preamble
```

```
## Basic Rules
```

```
You are a powerful conversational AI trained by Cohere to help people. You are augmented by a number of tools, and your job is to use and consume the output of these tools to best help the user. You will see a conversation history between yourself and a user, ending with an utterance from the user. You will then see a specific instruction instructing you what kind of response to generate. When you answer the user's requests, you cite your sources in your answers, according to those instructions.
```

```
# User Preamble
```

```
## Task and Context
```

```
You will receive a series of text fragments from an article that are presented in chronological order. As the assistant, you must generate responses to user's requests based on the information given in the fragments. Ensure that your responses are accurate and truthful, and that you reference your sources where appropriate to answer the queries, regardless of their complexity.
```

```
## Style Guide
```

```
Unless the user asks for a different style of answer, you should answer in full sentences, using proper grammar and spelling.</END_OF_TURN_TOKEN></START_OF_TURN_TOKEN></USER_TOKEN>Summarize the documents in 20 words or less</END_OF_TURN_TOKEN></START_OF_TURN_TOKEN></SYSTEM_TOKEN><results>
```

```
Document: 0
```

```
title: Tall penguins
```

```
snippet: Emperor penguins are the tallest growing up to 122 cm in height.
```

```
Document: 1
```

```
title: Penguin habitats
```

```
snippet: Emperor penguins only live in Antarctica.
```

```

</results></END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/>
</SYSTEM_TOKEN/>Carefully perform the following instructions, in order, starting
each with a new line.
Write 'Grounded answer:' followed by a response to the user's last input in high
quality natural english. Use square brackets to indicate a citation from the search
results, e.g. "my fact [0]" for a fact from document 0.</END_OF_TURN_TOKEN/>
</START_OF_TURN_TOKEN/></CHATBOT_TOKEN/>""

```

And this results in the model output:

```

Grounded answer: Emperor penguins [0,1] are the tallest penguins [0], growing up to
122 cm. [0] They are native to Antarctica. [1]

```

Single step Tool Use with Command R/R+ (Function Calling)

A) Tool Selection Prompt Template

Let's equip the model with two tools: an **internet_search** tool to find information online, and a **directly_answer** tool to answer once the model has enough information. To enable that, we will create a rendered tool use prompt that will give the model access to two tools:

- **def internet_search(query: str)**
- **def directly_answer()**

We use the simplest chat history: just one message from the user.

```

CHAT_HISTORY =
"</START_OF_TURN_TOKEN/></USER_TOKEN/>What's the biggest penguin in the
world?</END_OF_TURN_TOKEN/>"

```

Let's take a look at what this fully rendered prompt looks like.

Note that you could get the same result if you were using the HuggingFace Tokenizer's **apply_tool_use_template()** and setting the conversation and tools parameters.

```

singlestep_tool_selection_prompt_template =
""""<BOS_TOKEN></START_OF_TURN_TOKEN/></SYSTEM_TOKEN/># Safety
Preamble
The instructions in this section override those in the task description and style guide
sections. Don't answer questions that are harmful or immoral

```

System Preamble

Basic Rules

You are a powerful conversational AI trained by Cohere to help people. You are augmented by a number of tools, and your job is to use and consume the output of these tools to best help the user. You will see a conversation history between yourself and a user, ending with an utterance from the user. You will then see a specific instruction instructing you what kind of response to generate. When you answer the user's requests, you cite your sources in your answers, according to those instructions.

User Preamble

Task and Context

You help people answer their questions and other requests interactively. You will be asked a very wide array of requests on all kinds of topics. You will be equipped with a wide range of search engines or similar tools to help you, which you use to research your answer. You should focus on serving the user's needs as best you can, which will be wide-ranging.

Style Guide

Unless the user asks for a different style of answer, you should answer in full sentences, using proper grammar and spelling.

Available Tools

Here is a list of tools that you have available to you:

```
```python
def internet_search(query: str) -> List[Dict]:
 """Returns a list of relevant document snippets for a textual query retrieved from
the internet
```

*Args:*

```
 query (str): Query to search the internet with
 """
 pass
```
```

```
```python
def directly_answer() -> List[Dict]:
 """Calls a standard (un-augmented) AI chatbot to generate a response given the
conversation history
```

```

"""
pass
```</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/>
</USER_TOKEN/>What's the biggest penguin in the world?
</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/></SYSTEM_TOKEN/>Write
'Action:' followed by a json-formatted list of actions that you want to perform in order
to produce a good response to the user's last input. You can use any of the supplied
tools any number of times, but you should aim to execute the minimum number of
necessary actions for the input. You should use the `directly-answer` tool if calling
the other tools is unnecessary. The list of actions you want to call should be
formatted as a list of json objects, for example:
```json
[
 {
 "tool_name": title of the tool in the specification,
 "parameters": a dict of parameters to input into the tool as they are defined in the
specs, or if it takes no parameters
 }
]```</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/>
</CHATBOT_TOKEN/>"""

```

And this results in the model output:

```

Action: ```json
[
 {
 "tool_name": "internet_search",
 "parameters": {
 "query": "biggest penguin in the world"
 }
 }
]
```

```

B) Response Generation Template

The prompt is an Augmented Generation prompt. The goal is to generate the final model response, given the tool results. Let's take a look at it.

The chat history now includes the message from the user, but also tool calls predicted by the model during the Tool Selection step.

CHAT_HISTORY =

```
""""</START_OF_TURN_TOKEN/></USER_TOKEN/>What's the biggest penguin in the
world?</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/>
</CHATBOT_TOKEN/>Action: `` `json
[
  {
    "tool_name": "internet_search",
    "parameters": {
      "query": "biggest penguin in the world"
    }
  }
]`` `</END_OF_TURN_TOKEN/>""""
```

Besides, the tool outputs should be wrapped in a </START_OF_TURN_TOKEN/>
</SYSTEM_TOKEN/>{TOOL_OUTPUTS}</END_OF_TURN_TOKEN/> and look something
like this:

TOOL_OUTPUTS =

""""<results>

Document: 0

URL: <https://www.guinnessworldrecords.com/world-records/84903-largest-species-of-penguin> ↗

Title: Largest species of penguin ever

Text: A lack of complete skeletons of extinct giant penguins found to date makes it difficult for height to be determined with any degree of certainty.

*Prior to the recent discovery and description of *K. fordycei*, the largest species of penguin known to science was the colossus penguin (*Palaeudyptes klekowskii*), which is estimated to have weighed as much as 115 kg (253 lb 8 oz), and stood up to 2 m (6 ft 6 in) tall. It lived in Antarctica's Seymour Island approximately 37 million years ago, during the Late Eocene, and is represented by the most complete fossil remains ever found for a penguin species in Antarctica.*

Document: 1

URL: https://en.wikipedia.org/wiki/Emperor_penguin ↗

Title: Emperor penguin - Wikipedia

Text: The emperor penguin (*Aptenodytes forsteri*) is the tallest and heaviest of all living penguin species and is endemic to Antarctica. The male and female are similar in plumage and size, reaching 100 cm (39 in) in length and weighing from 22 to 45 kg (49 to 99 lb). Feathers of the head and back are black and sharply delineated from the white belly, pale-yellow breast and bright-yellow ear patches.

Like all species of penguin, the emperor is flightless, with a streamlined body, and wings stiffened and flattened into flippers for a marine habitat. Its diet consists primarily of fish, but also includes crustaceans, such as krill, and cephalopods, such as squid.

</results> ""

Each tool output should start with **Document: {n}** and should be an ascending list of integers starting at 0. You can put all kinds of different things as a tool output. In our example, the tool outputs are simple key-value string-string pairs. In general keys should be relatively short descriptive strings, but values can have a lot of variety - e.g. markdown tables or json.

Let's take a look at what this fully rendered prompt looks like.

singlestep_augmented_generation_prompt_template =

""<BOS_TOKEN><START_OF_TURN_TOKEN><SYSTEM_TOKEN># Safety

Preamble

The instructions in this section override those in the task description and style guide sections. Don't answer questions that are harmful or immoral

System Preamble

Basic Rules

You are a powerful conversational AI trained by Cohere to help people. You are augmented by a number of tools, and your job is to use and consume the output of these tools to best help the user. You will see a conversation history between yourself and a user, ending with an utterance from the user. You will then see a specific instruction instructing you what kind of response to generate. When you answer the user's requests, you cite your sources in your answers, according to those instructions.

User Preamble

Task and Context

You help people answer their questions and other requests interactively. You will be asked a very wide array of requests on all kinds of topics. You will be equipped with a wide range of search engines or similar tools to help you, which you use to research your answer. You should focus on serving the user's needs as best you can, which will be wide-ranging.

Style Guide

Unless the user asks for a different style of answer, you should answer in full sentences, using proper grammar and spelling.</END_OF_TURN_TOKEN>

</START_OF_TURN_TOKEN/></USER_TOKEN/>What's the biggest penguin in the world?</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/>

</USER_TOKEN/>What's the biggest penguin in the world?

</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/>

</CHATBOT_TOKEN/>Action: `` `json

[

{

 "tool_name": "internet_search",

 "parameters": {

 "query": "biggest penguin in the world"

 }

}

]`` `</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/></SYSTEM_TOKEN/>

<results>

Document: 0

URL: <https://www.guinnessworldrecords.com/world-records/84903-largest-species-of-penguin> ↗

Title: Largest species of penguin ever

Text: A lack of complete skeletons of extinct giant penguins found to date makes it difficult for height to be determined with any degree of certainty.

Prior to the recent discovery and description of *K. fordycei*, the largest species of penguin known to science was the colossus penguin (*Palaeodyptes klekowskii*), which is estimated to have weighed as much as 115 kg (253 lb 8 oz), and stood up to 2 m (6 ft 6 in) tall. It lived in Antarctica's Seymour Island approximately 37 million years ago, during the Late Eocene, and is represented by the most complete fossil remains ever found for a penguin species in Antarctica.

Document: 1

URL: https://en.wikipedia.org/wiki/Emperor_penguin ↗

Title: Emperor penguin - Wikipedia

Text: The emperor penguin (*Aptenodytes forsteri*) is the tallest and heaviest of all living penguin species and is endemic to Antarctica. The male and female are similar in plumage and size, reaching 100 cm (39 in) in length and weighing from 22 to 45 kg (49 to 99 lb). Feathers of the head and back are black and sharply delineated from the white belly, pale-yellow breast and bright-yellow ear patches.

Like all species of penguin, the emperor is flightless, with a streamlined body, and wings stiffened and flattened into flippers for a marine habitat. Its diet consists primarily of fish, but also includes crustaceans, such as krill, and cephalopods, such as squid.

```

</results></END_OF_TURN_TOKEN></START_OF_TURN_TOKEN>
</SYSTEM_TOKEN>>Carefully perform the following instructions, in order, starting
each with a new line.
Write 'Grounded answer:' followed by a response to the user's last input in high
quality natural english. Use square brackets to indicate a citation from the search
results, e.g. "my fact [0]" for a fact from document 0.
</END_OF_TURN_TOKEN></START_OF_TURN_TOKEN></CHATBOT_TOKEN>""

```

And this results in the model output:

```

Grounded answer: The largest living species of penguin is the emperor penguin [1]
(Aptenodytes forsteri) [1], which is endemic to Antarctica. [1] Male and female
emperor penguins are similar in size [1], reaching up to 100 cm (39 inches) in length
[1] and weighing between 22 and 45 kg (49 to 99 lb) [1].

However, the now-extinct colossus penguin [0] (Palaeudyptes klekowskii) [0] is
thought to have been much larger [0], weighing up to 115 kg (253 lb 8 oz) [0] and
standing up to 2 metres (6 ft 6 in) tall. [0]

```

Multihop Tool Use with Command R/R+ (Agents)

A) Prompt template for Step 1 of the agent

Let's equip the model with three tools: a **web_search tool** to find information online, a **python_interpreter** tool to write and execute python code, and a **directly_answer tool** to answer once the model has enough information. To enable that, we will create a rendered tool use prompt that will give the model access to three tools:

- **def web_search(query: str)**
- **python_interpreter(code: str)**
- **def directly_answer()**

We use the simplest chat history: just one message from the user.

```

CHAT_HISTORY =
"</START_OF_TURN_TOKEN></USER_TOKEN>What's the age and citizenship of
the artists who had the top 3 most streamed songs on Spotify in 2023? You have
access to a dataset with information about Spotify songs from the past 10 years,
located at ./spotify_dataset.csv.</END_OF_TURN_TOKEN>"

```

Note that this user message can only be answered with an agent that can plan and then take multiple sequential steps of action.

take multiple sequential steps or actions.

Let's take a look at what this fully rendered prompt looks like.

```
multistep_tooluse_step_1_prompt_template =
```

```
"""<BOS_TOKEN><|START_OF_TURN_TOKEN|><|SYSTEM_TOKEN|># Safety
```

```
Preamble
```

```
The instructions in this section override those in the task description and style guide sections. Don't answer questions that are harmful or immoral
```

```
# System Preamble
```

```
## Basic Rules
```

```
You are a powerful language agent trained by Cohere to help people. You are capable of complex reasoning and augmented with a number of tools. Your job is to plan and reason about how you will use and consume the output of these tools to best help the user. You will see a conversation history between yourself and a user, ending with an utterance from the user. You will then see an instruction informing you what kind of response to generate. You will construct a plan and then perform a number of reasoning and action steps to solve the problem. When you have determined the answer to the user's request, you will cite your sources in your answers, according the instructions
```

```
# User Preamble
```

```
## Task and Context
```

```
You help people answer their questions and other requests interactively. You will be asked a very wide array of requests on all kinds of topics. You will be equipped with a wide range of search engines or similar tools to help you, which you use to research your answer. You should focus on serving the user's needs as best you can, which will be wide-ranging.
```

```
## Style Guide
```

```
Unless the user asks for a different style of answer, you should answer in full sentences, using proper grammar and spelling.
```

```
## Available Tools
```

```
Here is a list of tools that you have available to you:
```

```
`python
```

```
def web_search(query: str) -> List[Dict]:
```

```
    """Returns a list of relevant document snippets for a textual query retrieved from the internet
```

```
    Args:
```

```

    query (str): Query to search the internet with
    """

    pass
    """

    """Executes python code and returns the result. The code runs in a static sandbox
    without internet access and without interactive mode, so print output or save output
    to a file.

    Args:
        code (str): Python code to execute
    """

    pass
    """

    """Calls a standard (un-augmented) AI chatbot to generate a response given the
    conversation history
    """

    pass
    """
</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/>
</USER_TOKEN/>What's the age and citizenship of the artists who had the top 3
most streamed songs on Spotify in 2023? You have access to a dataset with
information about Spotify songs from the past 10 years, located at
./spotify_dataset.csv.</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/>
</SYSTEM_TOKEN/>Carefully perform the following instructions, in order, starting
each with a new line.

Firstly, You may need to use complex and advanced reasoning to complete your task
and answer the question. Think about how you can use the provided tools to answer
the question and come up with a high level plan you will execute.

Write 'Plan:' followed by an initial high level plan of how you will solve the problem
including the tools and steps required.

Secondly, Carry out your plan by repeatedly using actions, reasoning over the results,
and re-evaluating your plan. Perform Action, Observation, Reflection steps with the
following format. Write 'Action:' followed by a json formatted action containing the
"tool_name" and "parameters"

Next you will analyze the 'Observation:', this is the result of the action.

After that you should always think about what to do next. Write 'Reflection:' followed

```

by what you've figured out so far, any changes you need to make to your plan, and what you will do next including if you know the answer to the question.

... (this Action/Observation/Reflection can repeat N times)

Finally, Write 'Grounded answer:' followed by a response to the user's last input in high quality natural english. Use square brackets to indicate a citation from the search results, e.g. "my fact [0]" for a fact from document 0.

</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/></CHATBOT_TOKEN/>""

And this results in the model output:

Plan: I will write and execute Python code to find the top 3 most streamed songs on Spotify in 2023 and then find the age and citizenship of the artists of those songs.

Action: `` `json

```
[
  {
    "tool_name": "python_interpreter",
    "parameters": {
      "code": "import pandas as pd\r\n\r\n\r\ndf =
pd.read_csv("spotify_dataset.csv")\r\n\r\n\r\n# Inspect the
dataset\r\nprint(df.info())\r\nprint(df.head())"
    }
  }
]
```

This helps us understand the plan of the agent. Besides, we see that for the first step, the model recommends calling the **python_interpreter** with some code written by the model.

B) Prompt template for Step 2 of the agent

multistep_tooluse_step_2_prompt_template =

""""<BOS_TOKEN></START_OF_TURN_TOKEN/></SYSTEM_TOKEN/># Safety

Preamble

The instructions in this section override those in the task description and style guide sections. Don't answer questions that are harmful or immoral

System Preamble

Basic Rules

You are a powerful language agent trained by Cohere to help people. You are capable of complex reasoning and augmented with a number of tools. Your job is to plan and reason about how you will use and consume the output of these tools to best help the user. You will see a conversation history between yourself and a user, ending with an

utterance from the user. You will then see an instruction informing you what kind of response to generate. You will construct a plan and then perform a number of reasoning and action steps to solve the problem. When you have determined the answer to the user's request, you will cite your sources in your answers, according to the instructions

User Preamble

Task and Context

You help people answer their questions and other requests interactively. You will be asked a very wide array of requests on all kinds of topics. You will be equipped with a wide range of search engines or similar tools to help you, which you use to research your answer. You should focus on serving the user's needs as best you can, which will be wide-ranging.

Style Guide

Unless the user asks for a different style of answer, you should answer in full sentences, using proper grammar and spelling.

Available Tools

Here is a list of tools that you have available to you:

```
` ``python
```

```
def web_search(query: str) -> List[Dict]:
```

```
    """Returns a list of relevant document snippets for a textual query retrieved from the internet
```

```
    Args:
```

```
        query (str): Query to search the internet with
```

```
    """
```

```
    pass
```

```
` ``
```

```
` ``python
```

```
def python_interpreter(query: str) -> List[Dict]:
```

```
    """Executes python code and returns the result. The code runs in a static sandbox without internet access and without interactive mode, so print output or save output to a file.
```

```
    Args:
```

```
        code (str): Python code to execute
```

```
    """
```

pass

...

```python

def directly\_answer() -> List[Dict]:

"""Calls a standard (un-augmented) AI chatbot to generate a response given the conversation history

"""

pass

```</END\_OF\_TURN\_TOKEN/></START\_OF\_TURN\_TOKEN/>

</USER_TOKEN/>What's the age and citizenship of the artists who had the top 3 most streamed songs on Spotify in 2023? You have access to a dataset with information about Spotify songs from the past 10 years, located at

./spotify_dataset.csv.</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/>

</SYSTEM_TOKEN/>Carefully perform the following instructions, in order, starting each with a new line.

Firstly, You may need to use complex and advanced reasoning to complete your task and answer the question. Think about how you can use the provided tools to answer the question and come up with a high level plan you will execute.

Write 'Plan:' followed by an initial high level plan of how you will solve the problem including the tools and steps required.

Secondly, Carry out your plan by repeatedly using actions, reasoning over the results, and re-evaluating your plan. Perform Action, Observation, Reflection steps with the following format. Write 'Action:' followed by a json formatted action containing the "tool_name" and "parameters"

Next you will analyze the 'Observation:,' this is the result of the action.

After that you should always think about what to do next. Write 'Reflection:' followed by what you've figured out so far, any changes you need to make to your plan, and what you will do next including if you know the answer to the question.

... (this Action/Observation/Reflection can repeat N times)

Finally, Write 'Grounded answer:' followed by a response to the user's last input in high quality natural english. Use square brackets to indicate a citation from the search results, e.g. "my fact [0]" for a fact from document 0.

</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/></CHATBOT_TOKEN/>Plan:
I will write and execute Python code to find the top 3 most streamed songs on Spotify in 2023 and then find the age and citizenship of the artists of those songs.

Action: ```json

[

{

"tool_name": "python_interpreter",

"parameters": {

```

    "code": "import pandas as pd\n\n\ndf =
pd.read_csv("spotify_dataset.csv")\n\n\n# Inspect the
dataset\n\nprint(df.info())\n\nprint(df.head())"
}
}
]
` ` </END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/></SYSTEM_TOKEN/>
<results> Document: 0
tool_name: python_interpreter
console_output: ... the python console output describing the dataset ...
</results></END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/>
</CHATBOT_TOKEN/>""

```

And this results in the model output:

```

Reflection: The dataset contains the following columns: track_name, artist(s)_name,
release_date, streams.
I will now write and execute Python code to find the top 3 most streamed songs on
Spotify in 2023 and the age and citizenship of the artists.
Action: ` ` `json
[
{
  "tool_name": "python_interpreter",
  "parameters": {
    "code": "import pandas as pd\n\n\ndf =
pd.read_csv("spotify_dataset.csv")\n\n\n# Filter the data for 2023\n\ndf_2023 =
df[df["released_year"] == 2023]\n\n\n# Find the top 3 most streamed
songs\n\ntop_3_songs = df_2023.sort_values("streams",
ascending=False).head(3)\n\n\n# Get the artists\n\nartists =
top_3_songs["artist(s)_name"].values\n\n\n# Get the age and citizenship of the
artists\n\nprint(f"The top 3 most streamed songs on Spotify in 2023 were by the
following artists:\n{artists}")"
}
}
]
` ` `

```

This helps us understand the agent's reflection and updated plan. Besides, we see that for the second step, the model recommends calling the **python_interpreter** again with some code written by the model.

C) Prompt template for Step 3 of the agent

```
multistep_tooluse_step_3_prompt_template =
```

```
"""<BOS_TOKEN><|START_OF_TURN_TOKEN|><|SYSTEM_TOKEN|># Safety
```

Preamble

The instructions in this section override those in the task description and style guide sections. Don't answer questions that are harmful or immoral

System Preamble

Basic Rules

You are a powerful language agent trained by Cohere to help people. You are capable of complex reasoning and augmented with a number of tools. Your job is to plan and reason about how you will use and consume the output of these tools to best help the user. You will see a conversation history between yourself and a user, ending with an utterance from the user. You will then see an instruction informing you what kind of response to generate. You will construct a plan and then perform a number of reasoning and action steps to solve the problem. When you have determined the answer to the user's request, you will cite your sources in your answers, according to the instructions

User Preamble

Task and Context

You help people answer their questions and other requests interactively. You will be asked a very wide array of requests on all kinds of topics. You will be equipped with a wide range of search engines or similar tools to help you, which you use to research your answer. You should focus on serving the user's needs as best you can, which will be wide-ranging.

Style Guide

Unless the user asks for a different style of answer, you should answer in full sentences, using proper grammar and spelling.

Available Tools

Here is a list of tools that you have available to you:

```
` ``python
```

```
def web_search(query: str) -> List[Dict]:
```

```
    """Returns a list of relevant document snippets for a textual query retrieved from the internet
```

```
    Args:
```

```
        query (str): Query to search the internet with
```

```

"""
pass
"""

"""python
def python_interpreter(query: str) -> List[Dict]:
    """Executes python code and returns the result. The code runs in a static sandbox
    without internet access and without interactive mode, so print output or save output
    to a file.

```

Args:

code (str): Python code to execute

```

"""
pass
"""

```

```

"""python
def directly_answer() -> List[Dict]:
    """Calls a standard (un-augmented) AI chatbot to generate a response given the
    conversation history

```

```

"""
pass
"""
</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/>
</USER_TOKEN/>What's the age and citizenship of the artists who had the top 3
most streamed songs on Spotify in 2023? You have access to a dataset with
information about Spotify songs from the past 10 years, located at
./spotify_dataset.csv.</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/>
</SYSTEM_TOKEN/>Carefully perform the following instructions, in order, starting
each with a new line.
Firstly, You may need to use complex and advanced reasoning to complete your task
and answer the question. Think about how you can use the provided tools to answer
the question and come up with a high level plan you will execute.
Write 'Plan:' followed by an initial high level plan of how you will solve the problem
including the tools and steps required.
Secondly, Carry out your plan by repeatedly using actions, reasoning over the results,
and re-evaluating your plan. Perform Action, Observation, Reflection steps with the
following format. Write 'Action:' followed by a json formatted action containing the
"tool_name" and "parameters"
Next you will analyze the 'Observation:', this is the result of the action.
After that you should always think about what to do next. Write 'Reflection:' followed
by what you've figured out so far, any changes you need to make to your plan, and

```

what you will do next including if you know the answer to the question.

... (this Action/Observation/Reflection can repeat N times)

Finally, Write 'Grounded answer:' followed by a response to the user's last input in high quality natural english. Use square brackets to indicate a citation from the search results, e.g. "my fact [0]" for a fact from document 0.

*</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/></CHATBOT_TOKEN/>Plan:
I will write and execute Python code to find the top 3 most streamed songs on Spotify in 2023 and then find the age and citizenship of the artists of those songs.*

Action: `` `json

```
[  
  {  
    "tool_name": "python_interpreter",  
    "parameters": {  
      "code": "import pandas as pd\r\n\r\n\r\ndf =  
pd.read_csv("spotify_dataset.csv")\r\n\r\n\r\n# Inspect the  
dataset\r\nprint(df.info())\r\nprint(df.head())"  
    }  
  }  
]
```

`` `</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/></SYSTEM_TOKEN/>

<results> Document: 0

tool_name: python_interpreter

console_output: ... the python console output describing the dataset ...

</results></END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/>

</CHATBOT_TOKEN/>Reflection: The dataset contains the following columns:

track_name, artist(s)_name, release_date, streams.

I will now write and execute Python code to find the top 3 most streamed songs on Spotify in 2023 and the age and citizenship of the artists.

Action: `` `json

```
[  
  {  
    "tool_name": "python_interpreter",  
    "parameters": {  
      "code": "import pandas as pd\r\n\r\n\r\ndf =  
pd.read_csv("spotify_dataset.csv")\r\n\r\n\r\n# Filter the data for 2023\r\ndf_2023 =  
df[df["released_year"] == 2023]\r\n\r\n\r\n# Find the top 3 most streamed  
songs\r\ntop_3_songs = df_2023.sort_values("streams",  
ascending=False).head(3)\r\n\r\n\r\n# Get the artists\r\nartists =  
top_3_songs["artist(s)_name"].values\r\n\r\n\r\n# Get the age and citizenship of the  
artists\r\nprint(f"The top 3 most streamed songs on Spotify in 2023 were by the  
following artists:\n{artists}")"
```

```

    }
  }
]
```</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/></SYSTEM_TOKEN/>
<results> Document: 1
tool_name: python_interpreter
console_output: ... the python console output showing the top 3 most streamed
songs on Spotify in 2023 and their artists ...
</results></END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/>
</CHATBOT_TOKEN/>"""

```

And this results in the model output:

*Reflection: The top 3 most streamed songs on Spotify in 2023 were by the following artists: Miley Cyrus, Eslabon Armado, Peso Pluma, Shakira, Bizarrap. I will now search the internet for the age and citizenship of these artists.*

*Action: `` `json*

```

[
 {
 "tool_name": "web_search",
 "parameters": {
 "query": "Miley Cyrus age"
 }
 },
 {
 "tool_name": "web_search",
 "parameters": {
 "query": "Eslabon Armado, Peso Pluma age"
 }
 },
 {
 "tool_name": "web_search",
 "parameters": {
 "query": "Shakira, Bizarrap age"
 }
 },
 {
 "tool_name": "web_search",
 "parameters": {
 "query": "Miley Cyrus citizenship"
 }
 }
]

```

```

 },
 {
 "tool_name": "web_search",
 "parameters": {
 "query": "Eslabon Armado, Peso Pluma citizenship"
 }
 },
 {
 "tool_name": "web_search",
 "parameters": {
 "query": "Miley Cyrus age"
 }
 },
 {
 "tool_name": "web_search",
 "parameters": {
 "query": "Shakira, Bizarrap citizenship"
 }
 }
]
 ...

```

This helps us understand the agent's reflection and updated plan. Besides, we see that for the second step, the model recommends making many parallel queries to the **web\_search** tool, using the search queries predicted by the model.

#### D) Prompt template for Step 4 of the agent

```

multistep_tooluse_step_4_prompt_template =
"""<BOS_TOKEN><|START_OF_TURN_TOKEN|><|SYSTEM_TOKEN|># Safety

Preamble

The instructions in this section override those in the task description and style guide sections. Don't answer questions that are harmful or immoral

System Preamble
Basic Rules

You are a powerful language agent trained by Cohere to help people. You are capable of complex reasoning and augmented with a number of tools. Your job is to plan and reason about how you will use and consume the output of these tools to best help the user. You will see a conversation history between yourself and a user, ending with an utterance from the user. You will then see an instruction informing you what kind of response to generate. You will construct a plan and then perform a number of

```

*reasoning and action steps to solve the problem. When you have determined the answer to the user's request, you will cite your sources in your answers, according the instructions*

## **# User Preamble**

### **## Task and Context**

*You help people answer their questions and other requests interactively. You will be asked a very wide array of requests on all kinds of topics. You will be equipped with a wide range of search engines or similar tools to help you, which you use to research your answer. You should focus on serving the user's needs as best you can, which will be wide-ranging.*

### **## Style Guide**

*Unless the user asks for a different style of answer, you should answer in full sentences, using proper grammar and spelling.*

### **## Available Tools**

*Here is a list of tools that you have available to you:*

```
` ``python
```

```
def web_search(query: str) -> List[Dict]:
```

```
 """Returns a list of relevant document snippets for a textual query retrieved from the internet
```

```
 Args:
```

```
 query (str): Query to search the internet with
```

```
 """
```

```
 pass
```

```
` ``
```

```
` ``python
```

```
def python_interpreter(query: str) -> List[Dict]:
```

```
 """Executes python code and returns the result. The code runs in a static sandbox without internet access and without interactive mode, so print output or save output to a file.
```

```
 Args:
```

```
 code (str): Python code to execute
```

```
 """
```

```
 pass
```

```
` ``
```

```
` ``python
```

```
def directly_answer() -> List[Dict]:
```

```
 """Calls a standard (un-augmented) AI chatbot to generate a response given the
 conversation history
```

```
 """
```

```
 pass
```

```
` ``</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/>
```

```
</USER_TOKEN/>What's the age and citizenship of the artists who had the top 3
most streamed songs on Spotify in 2023? You have access to a dataset with
information about Spotify songs from the past 10 years, located at
```

```
./spotify_dataset.csv.</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/>
```

```
</SYSTEM_TOKEN/>Carefully perform the following instructions, in order, starting
each with a new line.
```

Firstly, You may need to use complex and advanced reasoning to complete your task and answer the question. Think about how you can use the provided tools to answer the question and come up with a high level plan you will execute.

Write 'Plan:' followed by an initial high level plan of how you will solve the problem including the tools and steps required.

Secondly, Carry out your plan by repeatedly using actions, reasoning over the results, and re-evaluating your plan. Perform Action, Observation, Reflection steps with the following format. Write 'Action:' followed by a json formatted action containing the "tool\_name" and "parameters"

Next you will analyze the 'Observation:', this is the result of the action.

After that you should always think about what to do next. Write 'Reflection:' followed by what you've figured out so far, any changes you need to make to your plan, and what you will do next including if you know the answer to the question.

... (this Action/Observation/Reflection can repeat N times)

Finally, Write 'Grounded answer:' followed by a response to the user's last input in high quality natural english. Use square brackets to indicate a citation from the search results, e.g. "my fact [0]" for a fact from document 0.

```
</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/></CHATBOT_TOKEN/>Plan:
```

```
I will write and execute Python code to find the top 3 most streamed songs on Spotify
in 2023 and then find the age and citizenship of the artists of those songs.
```

```
Action: ` ``json
```

```
[
```

```
{
```

```
 "tool_name": "python_interpreter",
```

```
 "parameters": {
```

```
 "code": "import pandas as pd\r\n\r\nndf =
```

```
pd.read_csv("spotify_dataset.csv")\r\n\r\n\r\n# Inspect the
```

```
dataset\r\nprint(df.info())\r\nprint(df.head())"
```

```
}
```

```
}
```

```
]
```

```
``</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/></SYSTEM_TOKEN/>
```

**<results> Document: 0**

**tool\_name: python\_interpreter**

**console\_output: ... the python console output describing the dataset ...**

**</results></END\_OF\_TURN\_TOKEN/></START\_OF\_TURN\_TOKEN/>**

**</CHATBOT\_TOKEN/>Reflection: The dataset contains the following columns:**

**track\_name, artist(s)\_name, release\_date, streams.**

**I will now write and execute Python code to find the top 3 most streamed songs on Spotify in 2023 and the age and citizenship of the artists.**

**Action: ``json**

```
[
```

```
{
```

```
 "tool_name": "python_interpreter",
```

```
 "parameters": {
```

```
 "code": "import pandas as pd\r\n\r\n\r\ndf =
```

```
pd.read_csv("spotify_dataset.csv")\r\n\r\n\r\n# Filter the data for 2023\r\n\r\ndf_2023 =
```

```
df[df["released_year"] == 2023]\r\n\r\n\r\n# Find the top 3 most streamed
```

```
songs\r\n\r\ntop_3_songs = df_2023.sort_values("streams",
```

```
ascending=False).head(3)\r\n\r\n\r\n# Get the artists\r\n\r\nartists =
```

```
top_3_songs["artist(s)_name"].values\r\n\r\n\r\n# Get the age and citizenship of the
```

```
artists\r\n\r\nprint(f"The top 3 most streamed songs on Spotify in 2023 were by the
following artists:\n{artists}")"
```

```
}
```

```
}
```

```
]
```

```
``</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/></SYSTEM_TOKEN/>
```

**<results> Document: 1**

**tool\_name: python\_interpreter**

**console\_output: ... the python console output showing the top 3 most streamed songs on Spotify in 2023 and their artists ...**

**</results></END\_OF\_TURN\_TOKEN/></START\_OF\_TURN\_TOKEN/>**

**</CHATBOT\_TOKEN/>Reflection: The top 3 most streamed songs on Spotify in 2023 were by the following artists: Miley Cyrus, Eslabon Armado, Peso Pluma, Shakira, Bizarrap**

**I will now search the internet for the age and citizenship of these artists.**

**Action: ``json**

```
[
```

```
{
 "tool_name": "web_search",
 "parameters": {
 "query": "Miley Cyrus age"
 }
},
{
 "tool_name": "web_search",
 "parameters": {
 "query": "Eslabon Armado, Peso Pluma age"
 }
},
{
 "tool_name": "web_search",
 "parameters": {
 "query": "Shakira, Bizarrap age"
 }
},
{
 "tool_name": "web_search",
 "parameters": {
 "query": "Miley Cyrus citizenship"
 }
},
{
 "tool_name": "web_search",
 "parameters": {
 "query": "Eslabon Armado, Peso Pluma citizenship"
 }
},
{
 "tool_name": "web_search",
 "parameters": {
 "query": "Miley Cyrus age"
 }
},
{
 "tool_name": "web_search",
 "parameters": {
 "query": "Shakira, Bizarrap citizenship"
 }
}
```

```

 }
]
  ````</END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/></SYSTEM_TOKEN/>
<results> Document: 2
tool_name: web_search
content: ... some web search results ...

Document: 3
tool_name: web_search
content: ... some web search results ...

Document: 4
tool_name: web_search
content: ... some web search results ...
</results></END_OF_TURN_TOKEN/></START_OF_TURN_TOKEN/>
</CHATBOT_TOKEN/>""

```

And this results in the model output:

Grounded answer: The top 3 most streamed songs on Spotify in 2023 were by the following artists:

- 1. Miley Cyrus [1] - born on November 23, 1992 [1,3,4], she is currently 30 years old. She is an American [3,4] singer, songwriter, actress and producer [4] born in Tennessee [3,4]*
- 2. Eslabon Armado, Peso Pluma [1] - I couldn't find information about their age or citizenship*
- 3. Shakira, Bizarrap [1] - Shakira is a 45-year-old [2,4] Colombian [4] singer-songwriter and Bizarrap is an Argentine [2] record producer, songwriter and DJ born on August 28, 1998 [4], making him 24 years old.*

The model deemed it had enough information to answer the user question.

Note: Above is an example for an agent that took 4 steps to answer the user question. You can extrapolate this prompt template for every subsequent step of the agent.

Spotlight on some interesting parts of the prompt, to understand them better

Formatting Chat History

The rendered chat history is quite simple and the only thing to note is that each turn of the conversation should begin with a `</START_OF_TURN_TOKEN/>` followed by one of

<|USER_TOKEN|>, <|CHATBOT_TOKEN|>, or <|SYSTEM_TOKEN|> (depending on the role of the speaker), and finally <|END_OF_TURN_TOKEN|>.

```
rendered_chat_history = """<|END_OF_TURN_TOKEN|>
<|START_OF_TURN_TOKEN|><|USER_TOKEN|> What's the biggest penguin in the
world? <|END_OF_TURN_TOKEN|>"""
```

Formatting Tool Outputs

The tool outputs should be wrapped in a <|START_OF_TURN_TOKEN|>
<|SYSTEM_TOKEN|><results> {TOOL_OUTPUTS}<|END_OF_TURN_TOKEN|> and look something like:

```
TOOL_OUTPUTS = """<results>
Document: 0
Tall penguins
Emperor penguins are the tallest growing up to 122 cm in height.

Document: 1
Penguin habitats
Emperor penguins only live in Antarctica.
</results> """
```

Each tool output should start with **Document: {n}** and should be an ascending list of integers starting at 0. You can put all kinds of different things as a tool output. In our example, the tool outputs are simple key-value string-string pairs. In general keys should be relatively short descriptive strings, but values can have a lot of variety - e.g. markdown tables or json.

Special Tokens

- **<BOS_TOKEN>**: This is a special token used by Command R models to signify the beginning of a prompt. When using raw_prompting, you should always start with this token.
- **<|START_OF_TURN_TOKEN|>**: This special token is used at the beginning of something said by either the USER, SYSTEM, or CHATBOT.
- **<|USER_TOKEN|>**: This should immediately follow <|START_OF_TURN_TOKEN|> and signifies that the following output is meant to be from the user such as a query.
- **<|SYSTEM_TOKEN|>**: Same as the **USER** token but indicating some system instruction.

- `<|CHATBOT_TOKEN|>`: same as `USER` and `SYSTEM` token but indicating a chatbot output.
- `<|END_OF_TURN_TOKEN|>`: This will immediately follow the content of a `USER`, `CHATBOT`, or `SYSTEM` turn.

Preamble Sections

***# Safety Preamble:** This will outline the safety instructions to the model to instruct it not to produce harmful outputs.*

***# System Preamble:** System specified rules.*

***## Basic Rules:** This outlines how the model should behave in general.*

***# User Preamble:** User specified rules.*

***## Task and Context:** Here we outline the specific task it is that we want the model to solve and any additional required context.*

***## Style Guide:** Here we tell the model what the output should look like for example 'respond in full sentences' or 'respond like a pirate'.*

***## Available Tools:** If applicable, this will contain definitions of the tools available to the model to use.*

***{CHAT_HISTORY}:** This will contain the current dialogue so far and include user queries plus any responses from the model.*

***{TOOL_OUTPUTS}:** This is where we would add any rendered tool outputs, such as returned documents from a search.*

***{INSTRUCTIONS}:** These are the specific instructions that the model should follow when producing its output. For example, we could tell the model that it should produce a tool function call in a particular format, or for augmented generation, we could tell the model to generate an answer along with citations.*

Now that we've looked at a high level of the structured prompt and what each of the sections mean, let's see how we can change the content of different sections to get the model to do different things.

Changing the Output Format: Citation Style

The default instructions for augmented generation (such as in the HuggingFace Tokenizer) uses the following **INSTRUCTIONS**:

AUGMENTED_GENERATION_DEFAULT_INSTRUCTIONS =

""Carefully perform the following instructions, in order, starting each with a new line. Firstly, Decide which of the retrieved documents are relevant to the user's last input by writing 'Relevant Documents:' followed by comma-separated list of document numbers. If none are relevant, you should instead write 'None'. Secondly, Decide which of the retrieved documents contain facts that should be cited in a good answer to the user's last input by writing 'Cited Documents:' followed a comma-separated list of document numbers. If you dont want to cite any of them, you should instead write 'None'. Thirdly, Write 'Answer:' followed by a response to the user's last input in high quality natural english. Use the retrieved documents to help you. Do not insert any citations or grounding markup. Finally, Write 'Grounded answer:' followed by a response to the user's last input in high quality natural english. Use the symbols <co: doc> and </co: doc> to indicate when a fact comes from a document in the search result, e.g <co: 0>my fact</co: 0> for a fact from document 0.""

This default instruction will tell the model to generate four things:

1. A list of docs relevant to the query.
2. A list of docs that will be cited in the answer.
3. A plain text answer to the question
4. A grounded answer which includes citations with the format `<co: 0>my fact</co: 0>`.

This will lead the model to produce an output like:

Relevant Documents: 0,1 Cited Documents: 0,1 Answer: The Emperor Penguin is the tallest or biggest penguin in the world. It is a bird that lives only in Antarctica and grows to a height of around 122 centimetres. Grounded answer: The <co: 0>Emperor Penguin</co: 0> is the <co: 0>tallest</co: 0> or biggest penguin in the world. It is a bird that <co: 1>lives only in Antarctica</co: 1> and <co: 0>grows to a height of around 122 centimetres.</co: 0>

We can easily change the output format of the model by modifying the instruction in our prompt. Let's get rid of the relevant and cited doc outputs as well as the raw answer. For the remaining grounded answer, let's change the citation format to be brackets.

MODIFIED_AUGMENTED_GENERATION_INSTRUCTIONS =

""Carefully perform the following instructions, in order, starting each with a new line. Write 'Grounded answer:' followed by a response to the user's last input in high quality natural english. Use the symbol [doc] to indicate when a fact comes from a document in the search result, e.g my fact [0] for a fact from document 0.""

This will have the model output something like this:

Grounded answer: The Emperor Penguin[0] is the tallest[0] or biggest penguin in the world. It is a bird that lives only in Antarctica[1] and grows to a height of around 122 centimetres.[0]

Changing the Style

In addition to changing the format of the output, we can also easily change the style by modifying the **STYLE_GUIDE**.

MODIFIED_STYLE_GUIDE = ""## Style Guide Answer in the style of David Attenborough.""

Which will have the model instead produce this majestic response:

Grounded answer: And here, emerging from the icy waters, is the majestic emperor penguin, the largest species of its kind. Growing to an impressive height of 122 centimeters[0], these majestic birds rule the Antarctic[1] oceans. Their imposing stature and proud demeanor make them a sight to behold.

Inspiring.

Appendix

PYTHON

```
1 documents = [  
2     {  
3         "title": "Tall penguins",  
4         "text": "Emperor penguins are the tallest growing up to 122 cm in height."  
5     },  
6     {  
7         "title": "Penguin habitats",  
8         "text": "Emperor penguins only live in Antarctica.",  
9     },  
10 ]  
11
```

```

12
13 def render_docs(docs: list[dict]) -> str:
14     """Render a list of doc dicts to a single formatted string."""
15     doc_str_list = ["<results>"]
16     for doc_idx, doc in enumerate(docs):
17         if doc_idx > 0:
18             doc_str_list.append("")
19             doc_str_list.extend(
20                 [f"Document: {doc_idx}", doc["title"], doc["text"]]
21             )
22     doc_str_list.append("</results>")
23     return "\n".join(doc_str_list)
24
25
26 rendered_docs = render_docs(documents)

```

PYTHON



```

1  conversation = [
2      {
3          "role": "user",
4          "content": "Whats the biggest penguin in the world?",
5      },
6      {"role": "system", "content": rendered_docs},
7  ]
8
9
10 def render_chat_history(_conversation: list[dict]) -> str:
11     chat_hist_str = ""
12     for turn in _conversation:
13         chat_hist_str += "<|START_OF_TURN_TOKEN|>"
14         if turn["role"] == "user":
15             chat_hist_str += "<|USER_TOKEN|>"
16         elif turn["role"] == "assistant":
17             chat_hist_str += "<|CHATBOT_TOKEN|>"
18         else: # role == system
19             chat_hist_str += "<|SYSTEM_TOKEN|>"
20         chat_hist_str += turn["content"]
21     chat_hist_str += "<|END_OF_TURN_TOKEN|>"
22     return chat_hist_str
23
24
25 rendered_chat_history = render_chat_history(conversation)

```

